

Optymalizacja sieci MusicNN

Michał Wiliński, 188788

Szymon Zadworny, 188683

Spis treści

1	Realizowane zagadnienie	1
2	Wybór danych i modelu	2
2.1	Model	2
2.2	Zbiór danych	3
3	Zadanie docelowe i przygotowanie danych	3
4	Douczenie modelu odniesienia	4
4.1	Optymalizacja i przebieg treningu	5
4.2	Wyniki	5
5	Upraszczenie modelu	6
5.1	Algorytm RigL	7
6	Dodatkowe optymalizacje modelu	7
6.1	Klasteryzacja	7
6.1.1	Wpływ na warstwy splotowe	7
6.2	Kwantyzacja	8
7	Optymalizacja uczenia	9

1 Realizowane zagadnienie

Za cel wzięliśmy klasyfikowanie gatunku muzyki na podstawie mel-spektrogramu. W aplikacji telefonicznej mógłby służyć do analizy nagrania krótkiej części utworu, co pozwoliłoby użytkownikowi ocenić swoje gusta, oraz znaleźć inne zespoły grające w podobnym stylu. Oczekujemy że model po optymalizacji powinien zajmować mało miejsca na telefonie, oraz w miarę możliwości szybko zwracać wynik.

2 Wybór danych i modelu

2.1 Model

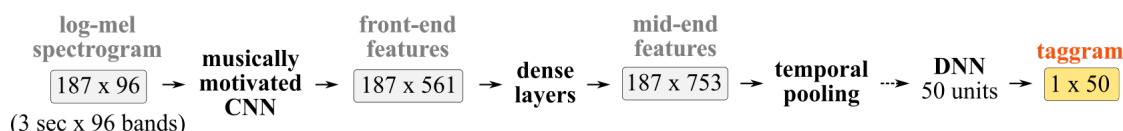
Jako podstawę naszego modelu wybraliśmy musicNN (<https://github.com/jordipons/musicnn>), publicznie dostępny model splotowy. Wyjściem modelu jest uporządkowanie 50 cech, obejmujących styl oraz instrumenty.

Tagi: guitar, classical, slow, techno, strings, drums, electronic, rock, fast, piano, ambient, beat, violin, vocal, synth, female, indian, opera, male, singing, vocals, no vocals, harpsichord, loud, quiet, flute, woman, male vocal, no vocal, pop, soft, sitar, solo, man, classic, choir, voice, new age, dance, male voice, female vocal, beats, harp, cello, no voice, weird, country, metal, female voice, choral.

Wyjściem z modelu są cechy ważne w określaniu gatunku, ale inne gatunki niż te określane przez nasz model. Dodatkowo jego rozmiar jest mały, co pasuje dla naszego docelowego zastosowania, oraz potrzeby wielokrotnego treningu z różnymi nastawami. W związku z tym model został punktem wyjścia do optymalizacji.

Wagi, na podstawie których chcemy transferować wiedzę, były wytrenowane na zbiorach MagnaTagATune. W repozytorium dostępne są również wagi wynikające z ćwiczenia na Million Song Dataset, z odpowiednimi tagami.

Ponieważ model powinien przyjmować dane o różnej długości, wejściowe nagrania dzielone jest na 3-sekundowe segmenty, z których tworzone są melspektrogramy o rozmiarach 187x96. Na wejście modelu podawany jest logarytm z ich wartości (rys. 1). Wyjścia o rozmiarze 1x50 można zagregować dla całego utworu i uśrednić.



Rysunek 1: Struktura modelu musicnn

Przy trenowaniu modelu na dostępnym sprzęcie, uzyskana została następująca charakterystyka:

Parametr	Wartość
Liczba parametrów	785245

Tabela 1: Porównanie modeli

2.2 Zbiór danych

Do wytrenowania wykorzystany został zbiór FMA (Free Music Archive) <https://github.com/mdeff/fma/tree/master> [2]

Posiada on kilka podzbiorów, oraz hierarchiczną strukturę gatunkową, co pozwoliło nam na pobranie najmniejszej wersji zbioru i przetestowanie modelu przed podjęciem decyzji o ostatecznych wielkościach zbiorów. Zbiór zgadza się z założeniami projektowymi, zawiera nagrania mp3, a był wykorzystywany (według autorów) w ponad 100 artykułach naukowych.

Inne zbiory danych wykorzystywane w muzyce to chociażby GTZAN, MagnaTagATune, Million Song Dataset, przedstawione w tabeli 2.

Zbiór	Liczba klas	Liczba nagrań	Uwagi
FMA-small	8 gatunków (hierarchicznych)	8000 (30s)	Dla FMA gatunki są ustawione w hierarchię drzewiastą, od najbardziej ogólnych do szczególnych. Tutaj najbardziej ogólne gatunki. Zbiór zrównoważony.
FMA-medium	16 gatunków	25000 (30s)	Gatunki najbardziej ogólne. Zbiór niezrównoważony.
FMA-large	161 gatunków	106574 (30s)	Zbiór niezrównoważony.
FMA-full	161 gatunków	106574	Nieprzycięte nagrania. Zbiór niezrównoważony
GTZAN [5]	10 gatunków	1000 (30s)	30-sekundowe nagrania
MagnaTagATune[4]	188 tagów	25877 segmentów (30s)	Do ewaluacji często używane 50 najczęstszych tagów [3].
Million Song Dataset (MSD)[1]	> 2000 tagów	1000000	Nie ma dostępnych nagrań, tylko cechy utworów. Do ewaluacji często używane 50 najczęstszych tagów [3].

Tabela 2: Porównanie muzycznych zbiorów danych

3 Zadanie docelowe i przygotowanie danych

Ponieważ klasy w zbiorze FMA są inne niż tagi wykorzystywane w oryginalnym zbiorze, nie zmieniamy ich. Podczas wykorzystania losowej próby części utworów i próbie przypisania ich do gatunków za pomocą słuchu, nie wykryliśmy żadnych gatunków, które myliłyby się ze sobą nawzajem. Test przeprowadziliśmy po

rekonstrukcji muzyki z melsprektrogramów, które stanowią wejście modelu.

Gatunki: International, Blues, Jazz, Classical, Old-Time / Historic, Country, Pop, Rock,

W celu porównania możliwości kiedy dane są zaszumione, do etykiet dodajemy szum symetryczny. 10% etykiet zmienia się w inną. Wykorzystane ziarna gwarantują powtarzalność.

W trakcie korzystania ze zbioru nie mogliśmy wykorzystać 6 z 6400 plików zbioru treningowego, ponieważ były niepoprawnie sformatowane. Nie dało się ich otworzyć z pomocą biblioteki librosa, ani w odtwarzaczu.

Z 30-sekundowych plików wydobyliśmy po 10 3-sekundowych melspektrogramów, które służyły nam za dane wejściowe, kilkukrotnie mniejszych rozmiarów niż oryginalne nagrania.

4 Douczanie modelu odniesienia

W oryginalnym modelu ostatnia warstwa stanowiła wyjście przypisujące embedding do klas. Jako embedding została więc wybrana przedostatnia warstwa, a cały oryginalny model został zamrożony.

```
embedding = orig_model.layers[-2].output
base_model = tf.keras.Model(
    inputs=orig_model.input,
    outputs=embedding
)
base_model.trainable = False
```

Następnie zdefiniowane zostały nowe warstwy Dense w celu umożliwienia trenowania modelu na nowych danych.

```
dense = tf.keras.layers.Dense(
    128,
    activation="relu",
    activity_regularizer=tf.keras.regularizers.L2(1e-5))(embedding)
dropout = tf.keras.layers.Dropout(0.5)(dense)
outputs = tf.keras.layers.Dense(
    num_classes_new,
    activation="softmax")(dropout)
modelv2 = tf.keras.Model(
    inputs=base_model.input,
    outputs=outputs
)
```

W ramach wyboru danych treningowych zostało zaszumione 25% całego zbioru danych.

```
train_ds, val_ds, test_ds = get_dataset(  
    augment_training=True,  
    all_noise_ratio=0.25  
)
```

Na tym etapie model nie był odmrażany - przeprowadzono tylko dotrenowanie dodatkowych warstw.

4.1 Optymalizacja i przebieg treningu

Dodana warstwa `Dense(128)` posiada regularyzację L2 w celu zapobiegnięcia overfittingu. Nie wybrano regularyzacji L1, ponieważ przerzedza ona daną warstwę. Planujemy przerzedzać resztę modelu i przerzedzanie 128 wag nie ma sensu w porównaniu z rozmiarem reszty.

Model został skompilowany z optymalizatorem Adam ze względu na różnorodność nowych danych i przestrzeni decyzyjnej. Optymalizatory SGD i RMSProp były testowane, lecz zbiegały się o wiele wolniej.

Jako funkcję straty wybrano sparse categorical cross-entropy ponieważ:

- mamy wiele etykiet
- nie są one kodowane jako one-hot vector

```
modelv2.compile(  
    optimizer="adam",  
    loss=keras.losses.SparseCategoricalCrossentropy(  
        from_logits=False  
    ),  
    metrics=['accuracy']  
)
```

Predykcją modelu nie jest tensor logitów ze względu na zastosowany w warstwie wyjściowej softmax.

4.2 Wyniki

Metryka	Dane
Czas uczenia pojedynczej epoki	~3 min
Dokładność	0.172
Rozmiar modelu	3.09 MB

5 Upraszczenie modelu

Ze względu na niską dokładność modelu przed upraszczaniem wybrane zostało upraszczanie nieustrukturyzowane. W ramach upraszczania modelu przeprowadzono również douczanie modelu na nowych danych. Najpierw odmrożono wszystkie warstwy poza warstwami BatchNormalization

```
def unfreeze(model):
    model.trainable = True
    for layer in model.layers:
        if isinstance(layer, tf.keras.layers.BatchNormalization):
            layer.trainable = False
```

Upraszczany model posiadał dziesiątki różnorodnych warstw, w tym wiele warstw stanowiących bezpośrednie operacje na tensorach oraz warstwy posiadające regularyzację. Następnie zdefiniowana została polityka upraszczania warstw:

```
def apply_pruning(layer):
    pruning_params = {
        'pruning_schedule': tfmot.sparsity.keras.ConstantSparsity(
            0.3,
            begin_step=0,
            frequency=100
        )
    }

    if isinstance(layer, tf.keras.layers.Conv2D) \
        or isinstance(layer, tf.keras.layers.Dense) \
        and layer.activity_regularizer is None:
        return prune_low_magnitude(layer, **pruning_params)

    return layer
```

Do upraszczania wybrane zostały warstwy Dense i Conv2D z pominięciem warstw poddanych regularyzacji. Jako scenariusz przerzedzania (*schedule*) zostało wybrane ConstantSparsity. Co 100 kroków 30% wag zostanie przerzedzona. Stopień przerzedzenia 0.3 został wybrany ze względu na niską dokładność bazowego modelu. W tym przypadku opcjonalne późniejsze ponowne przerzedzenie modelu jest bezpieczniejszym rozwiązaniem niż przerzedzanie go od razu do większego stopnia przerzedzenia.

Rozmiar modelu po kompresji XZ	2.3 MB
--------------------------------	--------

5.1 Algorytm RigL

W rozważanym problemie zastosowanie algorytmu RigL nie miało sensu - został on stworzony do trenowania rzadkich sieci od zera z dokładnością podobną do sieci które były przerzedzane po treningu. W naszym przypadku korzystamy w sieci, której zdecydowana większość wag była już wytrenowana wcześniej.

6 Dodatkowe optymalizacje modelu

6.1 Klasteryzacja

Tak samo jak w przypadku przerzedzania należy uważać na strukturę sieci neuronowej w ramach klasteryzacji. Ponownie zostały pominięte rodzaje warstw inne niż Conv2D i Dense. Wybrano 16 klastrów poszukiwanych za pomocą k-means.

```
def apply_clustering(layer):
    clustering_params = {
        'number_of_clusters': 16,
        'cluster_centroids_init': KMEANS_PLUS_PLUS,
        'preserve_sparsity': True
    }

    if isinstance(layer, tf.keras.layers.Conv2D) \
        or isinstance(layer, tf.keras.layers.Dense) \
        and layer.activity_regularizer is None:
        return cluster_weights(layer, **clustering_params)

    return layer
```

Metryka	Dane
Czas uczenia pojedynczej epoki	~3 min
Dokładność	0.164
Rozmiar modelu po kompresji XZ	0.35 MB

Dokładność modelu się pogorszyła, ponieważ pierwotny model nie był wytrenowany na dostatecznym zbiorze danych.

6.1.1 Wpływ na warstwy splotowe

Większość wykorzystywanego modelu stanowią warstwy splotowe. Każda z nich posiada szereg redundantnych filtrów, które wykrywają te same cechy. Same filtry

posiadają zaś lokalną strukturę opisującą wydobywaną przez niego cechę. Kwantyzacja wag pozwoli na dodatkowe zbliżenie tych struktur do siebie co pozwoli na jeszcze większą kompresję modelu. Jest to odrębne od kwantyzacji warstw pełnych, których rozproszona struktura globalna jest trudniejsza w kompresji.

6.2 Kwantyzacja

Nie byliśmy w stanie przeprowadzić kwantyzacji PCQAT ze względu na niekompatybilność modelu - wymusza on na nas korzystanie ze starszych bibliotek.

```
def quantize():
    stripped_clustered_model = tf.keras.saving.load_model(
        "stripped_clustered_model.keras"
    )

    quant_aware_annotate_model = tfmot.quantization.keras \
        .quantize_annotate_model(stripped_clustered_model)
    pcqat_model = tfmot.quantization.keras.quantize_apply(
        quant_aware_annotate_model,
        tfmot.experimental.combine \
            .Default8BitClusterPreserveQuantizeScheme(
                preserve_sparsity=True
            )) # Błąd wewnętrzny Keras

    pcqat_model.compile(
        optimizer=tf.keras.optimizers.Adam(
            learning_rate=1e-5
        ),
        loss=tf.keras.losses.SparseCategoricalCrossentropy(
            from_logits=False
        ),
        metrics=['accuracy']
    )

    train_ds, val_ds, test_ds = get_dataset(
        augment_training=True,
        all_noise_ratio=0.25
    )

    pcqat_model.fit(train_ds, epochs=2)
```



```
print(pcqat_model.evaluate(test_ds))
pcqat_model.save("pcqat_model.keras")
```

7 Optymalizacja uczenia

Jako że mamy do czynienia ze spektrogramami, augmentujemy nagrania przez:

- Przesunięcie w poprzek, z zawinięciem. Moment rozpoczęcia muzyki nie powinien wpływać na gatunek.
- Maskowanie spektrogramu na losowym paśmie częstotliwości - ma częściowo symulować nieoptymalne warunki nagraniowe, gdy niektóre częstotliwości są niewyraźne.

Bibliografia

- [1] Thierry Bertin-Mahieux i in. “The Million Song Dataset”. W: *Proceedings of the 12th International Society for Music Information Retrieval Conference (ISMIR 2011)*. 2011.
- [2] Michaël Defferrard i in. “FMA: A Dataset for Music Analysis”. W: *18th International Society for Music Information Retrieval Conference (ISMIR)*. 2017. arXiv: 1612.01840. URL: <https://arxiv.org/abs/1612.01840>.
- [3] Andres Ferraro i in. *How Low Can You Go? Reducing Frequency and Time Resolution in Current CNN Architectures for Music Auto-tagging*. 2020. arXiv: 1911.04824 [cs.IR]. URL: <https://arxiv.org/abs/1911.04824>.
- [4] Edith Law i in. “Evaluation of Algorithms Using Games: The Case of Music Tagging.” W: *sty*. 2009, s. 387–392.
- [5] George Tzanetakis i Perry Cook. “Musical Genre Classification of Audio Signals”. W: *IEEE Transactions on Speech and Audio Processing* 10 (sierp. 2002), s. 293–302. DOI: 10.1109/TSA.2002.800560.